

Sistema de Gestão de Territórios e Publicações (GTP)

Documento 07 – Arquitetura Frontend

Projeto: Sistema de Gestão de Territórios e Publicações (GTP)

Versão: 1.0.0

Status: Em Elaboração

Data: Julho/2026

Autor: Fabio André Zatta

Introdução e Arquitetura do Frontend

1 – Introdução

1.1 Objetivo

Este documento define a arquitetura técnica do frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu propósito é estabelecer os padrões arquiteturais, estruturais e de desenvolvimento da aplicação cliente, garantindo consistência, escalabilidade e facilidade de manutenção.

A arquitetura proposta visa atender aos requisitos funcionais, regras de negócio e casos de uso definidos nos documentos anteriores, proporcionando uma interface moderna, responsiva e acessível para os usuários do sistema.

1.2 Escopo

Este documento contempla:

- arquitetura da aplicação frontend;
- organização dos módulos;
- estrutura de diretórios;
- gerenciamento de estado;
- gerenciamento de dados remotos;
- comunicação com a API REST;
- autenticação;
- autorização;
- componentes reutilizáveis;
- Design System;
- roteamento;
- formulários;
- validações;
- internacionalização;
- acessibilidade;
- testes;
- otimização de desempenho.

Não serão detalhados aqui os contratos completos da API REST, que serão documentados no **Documento 10 – API REST**.

1.3 Tecnologias Adotadas

Categoria	Tecnologia
Linguagem	TypeScript 5.x
Framework	React 19
Build Tool	Vite
Estilização	Tailwind CSS
Componentes	Shadcn/UI (customizado)
Estado Global	Zustand
Dados Remotos	TanStack Query
Roteamento	React Router DOM
Formulários	React Hook Form
Validação	Zod
Requisições HTTP	Axios
Ícones	Lucide React
Testes Unitários	Vitest
Testes de Componentes	Testing Library
Testes E2E	Playwright
Qualidade	ESLint + Prettier

1.4 Objetivos Arquiteturais

A arquitetura do frontend foi projetada para atender aos seguintes objetivos:

Modularidade

Organizar a aplicação por funcionalidades, reduzindo acoplamento e facilitando a evolução do sistema.

Escalabilidade

Permitir a inclusão de novos módulos sem necessidade de reorganização estrutural.

Reutilização

Favorecer o uso de componentes compartilhados, hooks, utilitários e layouts reutilizáveis.

Testabilidade

Facilitar a criação de testes unitários, de integração e de interface.

Performance

Reduzir carregamentos desnecessários, otimizar renderizações e utilizar carregamento sob demanda (*lazy loading*).

Experiência do Usuário

Fornecer uma interface moderna, consistente, responsiva e acessível.

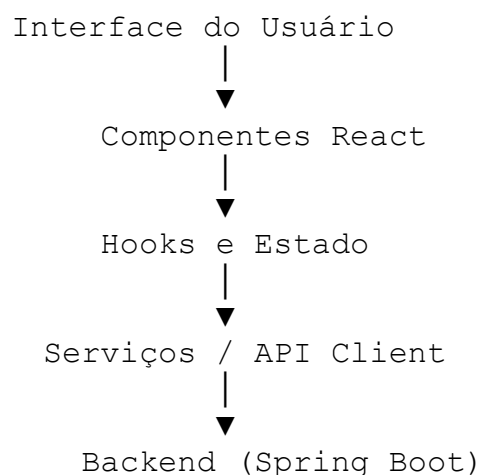
1.5 Princípios Arquiteturais

A implementação seguirá os seguintes princípios:

- separação de responsabilidades;
- baixo acoplamento;
- alta coesão;
- composição de componentes;
- tipagem forte com TypeScript;
- reutilização de código;
- consistência visual;
- arquitetura orientada a funcionalidades (*Feature-Based Architecture*).

1.6 Arquitetura Geral do Frontend

A aplicação será organizada em camadas funcionais.



Cada camada possui responsabilidades específicas, reduzindo dependências e facilitando a manutenção.

1.7 Arquitetura por Funcionalidades

O frontend será organizado por **features**, refletindo diretamente os módulos do domínio do GTP.

```
src/
├── features/
│   ├── auth/
│   ├── congregations/
│   ├── people/
│   ├── users/
│   ├── territories/
│   ├── publications/
│   ├── orders/
│   ├── campaigns/
│   ├── notifications/
│   ├── reports/
│   ├── settings/
│   └── audit/
```

Cada funcionalidade concentrará seus componentes, páginas, hooks, serviços e tipos, reduzindo dependências entre módulos.

1.8 Organização Geral da Aplicação

Além das funcionalidades, existirão diretórios compartilhados para elementos reutilizáveis.

```
src/
├── app/
├── assets/
├── components/
├── features/
├── hooks/
├── layouts/
├── lib/
├── providers/
├── routes/
├── services/
├── stores/
├── styles/
├── types/
└── utils/
```

Essa estrutura equilibra organização por domínio e reutilização de recursos comuns.

1.9 Integração com o Backend

Toda comunicação com o backend será realizada por meio da API REST definida no **Documento 10**.

As principais diretrizes são:

- utilização de HTTPS;
- autenticação via JWT;
- respostas padronizadas;
- tratamento centralizado de erros;
- cache de consultas com TanStack Query;
- tipagem completa das respostas.

Nenhum componente acessará diretamente a API sem passar pela camada de serviços.

1.10 Responsividade e Acessibilidade

Desde sua concepção, a aplicação deverá atender aos seguintes requisitos:

- design responsivo para desktop, tablet e dispositivos móveis;
- navegação por teclado;
- contraste adequado entre elementos;
- uso correto de atributos ARIA;
- suporte a leitores de tela;
- foco visível em elementos interativos;
- componentes acessíveis por padrão.

1.11 Convenções Gerais

Para manter a consistência do projeto, serão adotadas as seguintes convenções:

- componentes em **PascalCase**;
- hooks iniciados por `use`;
- arquivos de página terminados com `Page`;
- serviços terminados com `Service`;
- stores terminadas com `Store`;
- tipos centralizados quando compartilhados;
- nomenclatura em inglês para código-fonte.

2 – Estrutura do Projeto e Organização dos Módulos

2.1 Objetivo

Este capítulo define a organização física da aplicação frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer uma estrutura modular, padronizada e escalável, permitindo que o projeto evolua ao longo do tempo sem perda de organização.

A estrutura foi projetada para refletir o domínio da aplicação e facilitar a colaboração entre equipes de desenvolvimento.

2.2 Estrutura Geral do Projeto

O frontend será desenvolvido utilizando **React 19**, **TypeScript** e **Vite**, adotando uma arquitetura orientada a funcionalidades (*Feature-Based Architecture*).

Estrutura geral do projeto:

```
gtp-frontend/  
├── public/  
├── src/  
├── docs/  
├── scripts/  
├── .husky/  
├── .vscode/  
├── package.json  
├── tsconfig.json  
├── vite.config.ts  
├── tailwind.config.ts  
├── eslint.config.js  
├── prettier.config.js  
├── README.md  
└── .env
```

2.3 Diretório src

Todo o código-fonte da aplicação será concentrado no diretório src.

```
src/  
|  
├── app/  
├── assets/  
├── components/  
├── features/  
├── hooks/  
├── layouts/  
├── lib/  
├── providers/  
├── routes/  
├── services/  
├── stores/  
├── styles/  
├── types/  
├── utils/  
└── main.tsx
```

Cada diretório possui responsabilidades específicas.

2.4 Diretório app

Centraliza a configuração principal da aplicação.

```
app/  
|  
├── App.tsx  
├── AppProviders.tsx  
├── ErrorBoundary.tsx  
├── QueryProvider.tsx  
├── ThemeProvider.tsx  
└── AuthProvider.tsx
```

Responsabilidades:

- inicialização da aplicação;
- registro de providers globais;
- configuração do tema;

- tratamento de erros globais;
- contexto de autenticação;
- configuração do TanStack Query.

2.5 Diretório features

Cada funcionalidade do GTP será implementada em um módulo independente.

```
features/  
|  
├── auth/  
├── congregations/  
├── people/  
├── users/  
├── territories/  
├── publications/  
├── orders/  
├── campaigns/  
├── notifications/  
├── reports/  
├── settings/  
└── audit/
```

Cada feature conterá todos os elementos necessários para seu funcionamento.

2.6 Estrutura Interna de uma Feature

Exemplo utilizando o módulo **Territories**.

```
territories/  
|  
├── api/  
├── components/  
├── hooks/  
├── pages/  
├── services/  
├── store/  
├── types/  
├── utils/  
├── validators/  
└── index.ts
```

Essa estrutura favorece encapsulamento e baixo acoplamento entre funcionalidades.

2.7 Diretório components

Contém componentes reutilizáveis por toda a aplicação.

```
components/  
|  
├── ui/  
├── form/  
├── table/  
├── modal/  
├── layout/  
├── feedback/  
├── navigation/  
├── charts/  
└── shared/
```

Exemplos de componentes:

- Botões;
- Campos de formulário;
- Modais;
- Tabelas;
- Cards;
- Breadcrumbs;
- Paginação;
- Alertas;
- Toasts;
- Diálogos de confirmação.

2.8 Diretório layouts

Define os layouts utilizados pelas diferentes áreas da aplicação.

```
layouts/  
|  
├── MainLayout.tsx  
├── AuthLayout.tsx  
├── DashboardLayout.tsx  
├── ReportLayout.tsx  
└── EmptyLayout.tsx
```

Cada layout será responsável apenas pela estrutura visual, delegando o conteúdo às páginas.

2.9 Diretório routes

Centraliza a configuração do roteamento.

```
routes/  
|  
├── AppRoutes.tsx  
├── PrivateRoute.tsx  
├── PublicRoute.tsx  
├── RouteGuard.tsx  
└── index.ts
```

Responsabilidades:

- definição das rotas;
- proteção de rotas privadas;
- verificação de permissões;
- redirecionamentos.

2.10 Diretório services

Responsável pela comunicação com o backend.

```
services/
|
├── api.ts
├── auth.service.ts
├── territory.service.ts
├── publication.service.ts
├── order.service.ts
├── report.service.ts
└── user.service.ts
```

Todos os serviços utilizarão uma instância centralizada do Axios.

2.11 Diretório stores

Gerenciamento de estado global utilizando **Zustand**.

```
stores/
|
├── authStore.ts
├── appStore.ts
├── settingsStore.ts
├── notificationStore.ts
└── themeStore.ts
```

O estado global será utilizado apenas para informações compartilhadas.

2.12 Diretório hooks

Centraliza hooks reutilizáveis.

```
hooks/
|
├── useAuth.ts
├── useDebounce.ts
├── usePagination.ts
├── usePermissions.ts
├── useToast.ts
├── useTheme.ts
└── useWindowSize.ts
```

Hooks específicos de cada funcionalidade permanecerão dentro da respectiva feature.

2.13 Diretório types

Define contratos e tipos compartilhados.

```
types/  
|  
├── api.ts  
├── auth.ts  
├── pagination.ts  
├── user.ts  
├── territory.ts  
└── common.ts
```

Todos os tipos serão escritos em TypeScript.

2.14 Diretório utils

Funções auxiliares reutilizáveis.

```
utils/  
|  
├── formatDate.ts  
├── formatCurrency.ts  
├── validators.ts  
├── download.ts  
├── masks.ts  
└── permissions.ts
```

As funções deverão ser puras e independentes da interface.

2.15 Diretório styles

Centraliza estilos globais.

```
styles/
├──
├── globals.css
├── variables.css
├── typography.css
├── animations.css
└── themes.css
```

O Tailwind CSS será utilizado como base, complementado por estilos específicos quando necessário.

2.16 Diretório assets

Recursos estáticos utilizados pela aplicação.

```
assets/
├──
├── images/
├── icons/
├── logos/
├── fonts/
└── illustrations/
```

Esses recursos serão otimizados para reduzir o tempo de carregamento.

2.17 Arquivo index.ts

Cada feature disponibilizará um arquivo index.ts para exportação pública de seus componentes.

Exemplo:

```
features/
├── territories/
│   ├── components/
│   ├── pages/
│   ├── hooks/
│   └── index.ts
```

Essa abordagem simplifica importações e reduz o acoplamento entre módulos.

2.18 Organização por Domínio

A estrutura da aplicação refletirá diretamente os módulos do negócio.

Módulo	Feature
Autenticação	auth
Congregações	congregations
Pessoas	people
Usuários	users
Territórios	territories
Publicações	publications
Pedidos	orders
Campanhas	campaigns
Notificações	notifications
Relatórios	reports
Configurações	settings
Auditoria	audit

Essa correspondência facilita a rastreabilidade entre os documentos de requisitos e a implementação.

2.19 Convenções de Organização

Para manter a consistência do projeto, serão adotadas as seguintes diretrizes:

- uma feature não acessa diretamente arquivos internos de outra feature;
- componentes compartilhados permanecem em components;
- estado global restrito ao mínimo necessário;
- tipos específicos permanecem na própria feature;

- hooks reutilizáveis ficam em hooks;
- comunicação com a API ocorre exclusivamente pela camada de serviços;
- exportações públicas devem ser realizadas pelo arquivo index.ts.

2.20 Benefícios da Estrutura

A organização proposta proporciona:

- alta modularidade;
- facilidade de manutenção;
- reutilização de componentes;
- redução de dependências entre módulos;
- melhor organização do código;
- suporte ao crescimento do sistema;
- maior produtividade da equipe.

Perfeito. Vamos continuar mantendo o mesmo nível de detalhamento técnico adotado em toda a documentação.

3 – Arquitetura da Interface e Componentes

3.1 Objetivo

Este capítulo define a arquitetura da camada de interface do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer padrões para o desenvolvimento das telas da aplicação, promovendo:

- reutilização de componentes;
- consistência visual;
- baixo acoplamento;
- facilidade de manutenção;
- alta performance;

- acessibilidade.

Toda a interface será construída utilizando **React 19**, com componentes funcionais, **TypeScript** e composição de componentes.

3.2 Princípios da Arquitetura de Interface

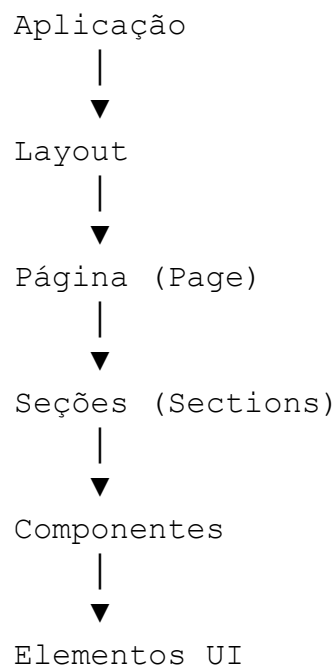
A interface seguirá os seguintes princípios:

- composição ao invés de herança;
- responsabilidade única para cada componente;
- separação entre lógica e apresentação;
- reutilização máxima;
- tipagem forte;
- componentes previsíveis;
- comunicação unidirecional.

Esses princípios reduzem a complexidade e facilitam a evolução da aplicação.

3.3 Hierarquia da Interface

A interface será organizada em diferentes níveis de abstração.



Cada nível possui responsabilidades específicas.

3.4 Organização dos Componentes

Os componentes serão classificados conforme sua finalidade.

```
components/  
├── ui/  
├── form/  
├── table/  
├── navigation/  
├── feedback/  
├── modal/  
├── charts/  
├── dashboard/  
├── layout/  
└── shared/
```

Essa classificação facilita a localização e reutilização.

3.5 Componentes Base (UI)

Os componentes da pasta ui representam os blocos fundamentais da interface.

Exemplos:

Button
Input
Textarea
Select
Checkbox
Switch
Badge
Avatar
Card
Tooltip
Dialog
Popover
Accordion
Tabs

- Separator
- Skeleton
- Spinner

Esses componentes deverão seguir o Design System do GTP.

3.6 Componentes de Formulário

Os formulários serão compostos por componentes especializados.

- form/
 - FormField
 - FormLabel
 - FormError
 - FormGroup
 - DatePicker
 - TimePicker
 - SearchInput
 - CurrencyInput
 - PhoneInput
 - PasswordInput

Esses componentes serão integrados ao **React Hook Form**.

3.7 Componentes de Tabela

Grande parte do sistema será baseada em consultas e listagens.

Estrutura proposta:

- table/
 - DataTable
 - TableToolbar
 - ColumnSelector
 - Pagination
 - TableFilter
 - TableSearch
 - EmptyState

As tabelas deverão suportar:

- paginação;
- ordenação;
- filtros;
- pesquisa;
- seleção múltipla;
- exportação.

3.8 Componentes de Navegação

Os elementos de navegação serão reutilizados em toda a aplicação.

navigation/

Sidebar

Topbar

Breadcrumb

Menu

Navbar

UserMenu

Pagination

Tabs

Esses componentes serão utilizados pelos layouts.

3.9 Componentes de Feedback

A comunicação com o usuário seguirá padrões consistentes.

feedback/

Alert

Toast

Snackbar

ConfirmDialog

Loading

ProgressBar

ErrorPage

SuccessMessage

Todos os feedbacks deverão possuir comportamento uniforme.

3.10 Componentes de Dashboard

O painel principal utilizará componentes específicos.

dashboard/

StatisticCard

ChartCard

RecentActivities

QuickActions

NotificationsWidget

CalendarWidget

SummaryCard

Esses componentes permitirão a construção de dashboards dinâmicos.

3.11 Componentes Compartilhados

A pasta shared reunirá componentes utilizados por diferentes módulos.

Exemplos:

- filtros;
- cabeçalhos;
- cards de informação;
- listas;
- seletor de congregação;
- seletor de período;
- componentes de auditoria.

3.12 Organização das Páginas

Cada funcionalidade possuirá suas próprias páginas.

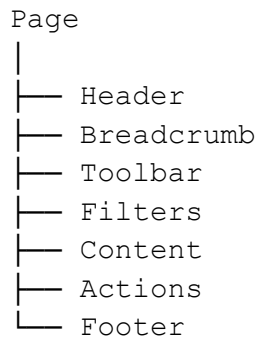
Exemplo:

```
territories/  
pages/  
TerritoryListPage  
TerritoryDetailsPage  
CreateTerritoryPage  
EditTerritoryPage  
ReserveTerritoryPage
```

As páginas deverão apenas compor componentes e orquestrar o fluxo da interface.

3.13 Estrutura de uma Página

Uma página será composta pelos seguintes elementos.

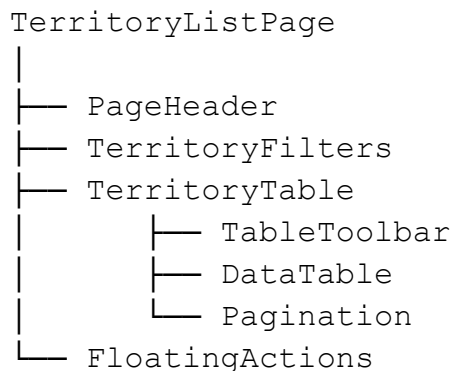


Essa estrutura será reutilizada sempre que aplicável.

3.14 Composição de Componentes

A composição seguirá uma abordagem hierárquica.

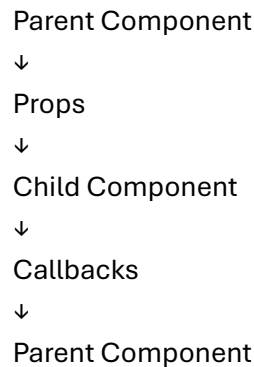
Exemplo:



Cada componente possuirá responsabilidades bem definidas.

3.15 Comunicação entre Componentes

A comunicação seguirá o fluxo natural do React.



Estados compartilhados utilizarão Context ou Zustand apenas quando necessário.

3.16 Estados dos Componentes

Cada componente deverá tratar explicitamente seus estados.

Exemplos:

- carregando;
- vazio;
- erro;
- sucesso;
- edição;
- somente leitura;
- desabilitado.

Essa padronização melhora a experiência do usuário.

3.17 Componentes Inteligentes e Apresentacionais

A arquitetura distinguirá dois tipos de componentes.

Componentes Apresentacionais

Responsáveis apenas pela renderização.

Características:

- recebem propriedades;
- não conhecem a API;
- reutilizáveis;
- sem regras de negócio.

Componentes Inteligentes

Responsáveis pela coordenação da interface.

Características:

- utilizam hooks;
- consultam a API;
- controlam estado;
- executam ações.

Essa separação melhora a reutilização e a testabilidade.

3.18 Estratégia de Renderização

A aplicação utilizará:

- renderização baseada em componentes;
- carregamento sob demanda (*lazy loading*);
- divisão de código (*code splitting*);
- memoização quando apropriado;
- suspense para carregamentos assíncronos.

Essas estratégias contribuem para o desempenho da aplicação.

3.19 Design Responsivo

Todos os componentes deverão ser responsivos.

Breakpoints propostos:

Dispositivo	Largura
Mobile	até 640 px
Tablet	641–1024 px
Notebook	1025–1440 px
Desktop	acima de 1440 px

A interface deverá adaptar menus, tabelas, formulários e painéis para diferentes resoluções, priorizando a usabilidade.

3.20 Acessibilidade

Todos os componentes deverão atender às diretrizes de acessibilidade.

Requisitos:

- navegação por teclado;
- foco visível;
- contraste adequado;
- atributos ARIA;
- mensagens compreensíveis;
- compatibilidade com leitores de tela;
- áreas clicáveis adequadas.

Sempre que possível, os componentes reutilizarão elementos HTML semânticos.

3.21 Tratamento de Erros na Interface

A interface deverá tratar erros de forma padronizada.

Exemplos:

- falha de conexão;
- autenticação expirada;
- erro de validação;
- recurso inexistente;
- operação não autorizada;
- indisponibilidade temporária do servidor.

As mensagens devem ser objetivas, indicar a ação necessária e evitar expor detalhes técnicos.

3.22 Relação com o Design System

Todos os componentes deverão utilizar exclusivamente os elementos definidos no Design System do GTP.

Isso garante:

- consistência visual;
- padronização de comportamento;
- facilidade de manutenção;
- evolução centralizada da identidade visual.

3.23 Organização por Funcionalidade

Cada módulo implementará sua própria composição de componentes.

Exemplo:

```
features/  
└─ territories/  
    └─ components/  
        └─ TerritoryCard.tsx  
        └─ TerritoryTable.tsx  
        └─ TerritoryFilters.tsx  
        └─ TerritoryStatusBadge.tsx  
        └─ TerritoryForm.tsx  
    └─ pages/  
    └─ hooks/  
    └─ index.ts
```

Essa organização mantém o encapsulamento das funcionalidades.

3.24 Benefícios da Arquitetura

A arquitetura baseada em componentes proporciona:

- reutilização de código;
- interfaces consistentes;
- desenvolvimento paralelo entre equipes;
- testes mais simples;
- redução de duplicidade;
- melhor desempenho;
- maior facilidade para evolução do sistema.

4 – Gerenciamento de Estado e Dados

4.1 Objetivo

Este capítulo define a estratégia de gerenciamento de estado e dados do frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer uma arquitetura previsível, escalável e eficiente para o tratamento das informações utilizadas pela aplicação, diferenciando claramente os estados locais, globais e remotos.

A solução adotada combina **Zustand** para o estado global da aplicação e **TanStack Query** para o gerenciamento de dados provenientes da API REST.

4.2 Princípios

O gerenciamento de estado seguirá os seguintes princípios:

- cada dado deve possuir uma única fonte de verdade (*Single Source of Truth*);
- estados locais permanecerão nos componentes;
- estados compartilhados utilizarão Zustand;
- dados da API serão gerenciados exclusivamente pelo TanStack Query;
- evitar duplicação de informações;
- minimizar renderizações desnecessárias;
- priorizar imutabilidade e previsibilidade.

4.3 Classificação dos Estados

Os estados da aplicação serão divididos em três categorias.

Tipo	Responsabilidade	Tecnologia
Estado Local	Controle da interface do componente	React Hooks
Estado Global	Dados compartilhados por toda a aplicação	Zustand
Estado Remoto	Dados obtidos da API REST	TanStack Query

Essa separação reduz o acoplamento e simplifica a manutenção.

4.4 Estado Local

O estado local será utilizado para controlar informações temporárias da interface.

Exemplos:

- abertura de modais;
- campos de pesquisa;
- abas selecionadas;
- expansão de painéis;
- ordenação temporária;
- filtros ainda não aplicados.

Esses estados utilizarão os hooks nativos do React (useState, useReducer, useMemo, useCallback).

4.5 Estado Global

O Zustand será utilizado apenas para informações compartilhadas por múltiplos módulos.

Exemplos:

- usuário autenticado;
- token de acesso;
- congregação ativa;
- preferências do usuário;
- idioma selecionado;
- tema da aplicação;
- notificações globais.

O estado global deverá permanecer enxuto, evitando armazenar dados que possam ser obtidos diretamente da API.

4.6 Estrutura das Stores

As stores serão organizadas por responsabilidade.

```
stores/
|
├── authStore.ts
├── appStore.ts
├── themeStore.ts
├── settingsStore.ts
├── notificationStore.ts
├── congregationStore.ts
└── userPreferencesStore.ts
```

Cada store será independente e responsável apenas pelo seu domínio.

4.7 Store de Autenticação

A authStore será responsável por manter o contexto da sessão do usuário.

Informações armazenadas:

- usuário autenticado;
- token JWT;
- permissões;
- perfis;
- data de expiração;
- estado de autenticação.

Essa store será inicializada durante o login e limpa no logout.

4.8 Store da Aplicação

A appStore controlará informações gerais da interface.

Exemplos:

- indicador global de carregamento;
- menu lateral expandido;
- idioma atual;

- breadcrumb dinâmico;
- título da página;
- configurações da sessão.

4.9 Estado Remoto

Todos os dados provenientes do backend serão gerenciados pelo **TanStack Query**.

Exemplos:

- usuários;
- territórios;
- publicações;
- pedidos;
- campanhas;
- notificações;
- relatórios.

Os componentes nunca deverão armazenar permanentemente esses dados em Zustand.

4.10 Organização das Queries

Cada funcionalidade possuirá suas próprias consultas.

Exemplo:

```
features/  
└─ territories/  
    └─ api/  
        └─ queries.ts  
        └─ mutations.ts  
        └─ keys.ts
```

Essa organização facilita a manutenção e evita dependências cruzadas.

4.11 Chaves de Cache (Query Keys)

As consultas utilizarão chaves padronizadas.

Exemplos:

```
territories
territories.list
territories.details
territories.assignments
users.list
users.details
orders.list
orders.details
```

As chaves deverão refletir a estrutura do domínio.

4.12 Cache de Dados

O TanStack Query será responsável por:

- armazenar resultados das consultas;
- reutilizar dados existentes;
- reduzir chamadas desnecessárias;
- sincronizar automaticamente informações.

O tempo de cache poderá variar conforme o tipo de dado.

4.13 Invalidação de Cache

Após operações de escrita, as consultas relacionadas deverão ser invalidadas.

Exemplos:

- criação de território;
- edição de usuário;
- devolução de território;

- entrega de pedido;
- atualização de estoque.

Essa estratégia garante que a interface apresente sempre informações consistentes.

4.14 Atualizações Otimistas

Operações simples poderão utilizar **Optimistic Updates**.

Exemplos:

- favoritar relatório;
- marcar notificação como lida;
- alterar preferências;
- atualizar status visual.

Caso a operação falhe, o estado anterior deverá ser restaurado automaticamente.

4.15 Paginação

As listagens utilizarão paginação baseada no servidor.

Informações controladas:

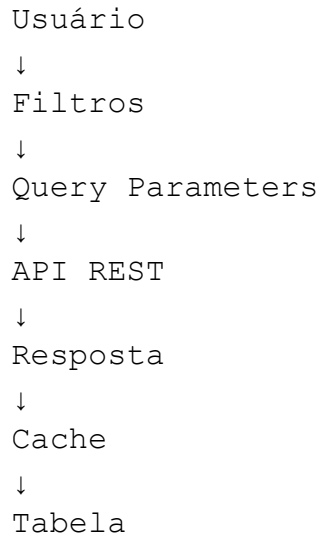
- página atual;
- quantidade de registros;
- ordenação;
- filtros;
- total de páginas.

O TanStack Query armazenará cada página em cache.

4.16 Filtros e Pesquisa

Filtros serão tratados separadamente dos dados.

Fluxo:



Essa abordagem evita inconsistências entre filtros e resultados.

4.17 Sincronização de Dados

A atualização automática poderá ocorrer:

- ao retornar para a aba do navegador;
- após reconexão com a internet;
- após invalidação manual;
- em intervalos configuráveis para módulos específicos.

A frequência de sincronização deverá considerar o impacto no desempenho.

4.18 Tratamento de Erros

As consultas deverão tratar erros de forma padronizada.

Tipos previstos:

- autenticação expirada;
- acesso negado;

- recurso inexistente;
- erro de validação;
- indisponibilidade do servidor;
- falha de rede.

Os erros serão apresentados ao usuário por meio dos componentes de feedback definidos no Capítulo 3.

4.19 Persistência de Preferências

Algumas informações poderão ser persistidas localmente.

Exemplos:

- tema claro ou escuro;
- idioma;
- tamanho das colunas das tabelas;
- filtros favoritos;
- layout do dashboard.

Esses dados poderão utilizar localStorage ou outra estratégia equivalente.

4.20 Fluxo Geral de Dados

```
Usuário
↓
Interface React
↓
Hooks
↓
TanStack Query
↓
API REST
↓
Spring Boot
↓
PostgreSQL
```

As respostas retornam pelo mesmo fluxo, sendo armazenadas em cache antes de chegarem aos componentes.

4.21 Relação entre Zustand e TanStack Query

As responsabilidades de cada tecnologia serão claramente separadas.

Zustand	TanStack Query
Sessão do usuário	Dados da API
Tema	Consultas REST
Idioma	Cache de respostas
Preferências	Mutações
Estado da interface	Paginação
Configurações locais	Sincronização

Essa divisão evita redundância e simplifica a arquitetura.

4.22 Benefícios da Estratégia

A arquitetura proposta proporciona:

- previsibilidade;
- menor consumo de memória;
- redução de chamadas HTTP;
- sincronização automática;
- melhor experiência do usuário;
- maior desempenho;
- facilidade para testes;
- escalabilidade.

4.23 Relação com os Demais Documentos

Este capítulo complementa:

Documento	Relação
Documento 02 – Requisitos Funcionais	Define os dados manipulados pela interface.
Documento 04 – Casos de Uso	Determina os fluxos de leitura e escrita.
Documento 05 – Arquitetura do Sistema	Estabelece a comunicação entre frontend e backend.
Documento 06 – Arquitetura Backend	Define os serviços consumidos pelo frontend.
Documento 10 – API REST	Especifica os contratos utilizados pelo TanStack Query.

5 – Comunicação com a API REST

5.1 Objetivo

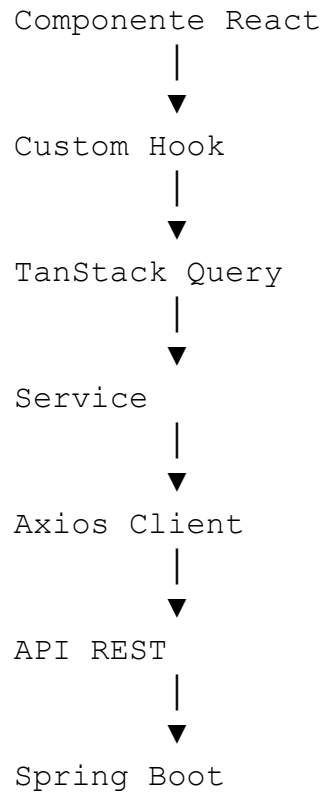
Este capítulo define a arquitetura de comunicação entre o frontend do **Gestor de Territórios e Publicações (GTP)** e o backend desenvolvido em **Spring Boot 3**.

Seu objetivo é estabelecer uma camada de comunicação padronizada, segura, desacoplada e reutilizável, permitindo que todos os módulos da aplicação consumam os serviços da API REST de maneira consistente.

A comunicação será realizada exclusivamente por HTTPS, utilizando requisições HTTP e respostas no formato JSON.

5.2 Arquitetura da Comunicação

A comunicação entre a interface e o backend seguirá uma arquitetura em camadas.



Cada camada possui responsabilidades específicas, reduzindo o acoplamento e facilitando testes e manutenção.

5.3 Cliente HTTP

Todas as requisições utilizarão uma única instância configurada do **Axios**.

Estrutura sugerida:

```
src/
├── services/
│   ├── api.ts
│   ├── interceptors.ts
│   ├── http.ts
│   └── endpoints.ts
```

Centralizar a configuração evita repetição e garante comportamento uniforme.

5.4 Configuração da Base URL

A URL da API será definida por variáveis de ambiente, permitindo diferentes configurações para desenvolvimento, homologação e produção.

Exemplo de organização:

```
.env.development
.env.staging
.env.production
```

Essa abordagem facilita a implantação em diferentes ambientes sem alterações no código-fonte.

5.5 Organização dos Serviços

Cada módulo da aplicação possuirá seus próprios serviços.

```
services/
|
├── auth.service.ts
├── user.service.ts
├── congregation.service.ts
├── territory.service.ts
├── publication.service.ts
├── order.service.ts
├── campaign.service.ts
├── notification.service.ts
├── report.service.ts
└── settings.service.ts
```

Cada serviço será responsável exclusivamente pelas operações relacionadas ao seu domínio.

5.6 Estrutura de um Serviço

Um serviço deverá encapsular todas as operações de comunicação com determinado recurso da API.

Exemplo de responsabilidades:

- listar registros;
- consultar por identificador;
- criar novos registros;
- atualizar informações;
- excluir registros;
- executar operações específicas do domínio.

A lógica de apresentação nunca deverá realizar chamadas HTTP diretamente.

5.7 Interceptadores de Requisição

Os interceptadores permitirão adicionar comportamentos comuns antes do envio das requisições.

Principais responsabilidades:

- incluir o token JWT no cabeçalho Authorization;
- definir cabeçalhos padrão;
- registrar informações para depuração (em ambiente de desenvolvimento);
- configurar tempo limite (*timeout*);
- adicionar identificadores de rastreamento, quando aplicável.

5.8 Interceptadores de Resposta

Após o recebimento das respostas, os interceptadores serão responsáveis por:

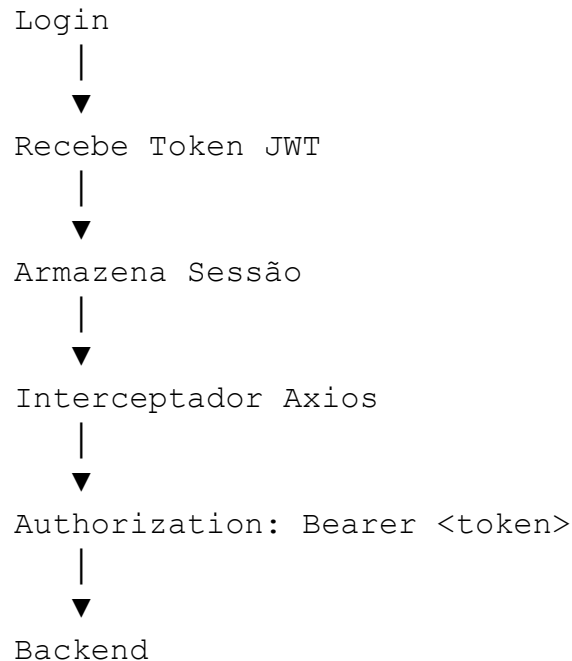
- tratar erros HTTP de forma centralizada;
- renovar a sessão, quando suportado;
- redirecionar para a tela de login em caso de autenticação expirada;
- padronizar mensagens de erro;
- registrar eventos relevantes para monitoramento.

Essa abordagem evita duplicação de tratamento em cada componente.

5.9 Autenticação nas Requisições

Todas as rotas protegidas deverão enviar o token JWT utilizando o padrão Bearer.

Fluxo:



O gerenciamento do ciclo de vida do token será centralizado na camada de autenticação.

5.10 Tratamento de Erros HTTP

Os códigos de status retornados pela API deverão ser tratados de forma uniforme.

Código		Significado	Ação no Frontend
400	Requisição inválida		Exibir erros de validação.
401	Não autenticado		Encerrar sessão e redirecionar para o login.
403	Acesso negado		Exibir mensagem de permissão insuficiente.
404	Recurso não encontrado		Exibir tela ou mensagem apropriada.
409	Conflito		Informar conflito de dados ao usuário.
422	Erro de validação de negócio		Exibir mensagens específicas do domínio.
500	Erro interno		Exibir mensagem genérica e registrar o erro.

Código	Significado	Ação no Frontend
503	Serviço indisponível	Informar indisponibilidade temporária.

5.11 Padronização das Respostas

Sempre que possível, a API retornará estruturas consistentes para facilitar o consumo pelo frontend.

As respostas deverão conter:

- dados da operação;
- informações de paginação (quando aplicável);
- mensagens de sucesso;
- detalhes de erros;
- identificador de rastreamento da requisição, quando disponível.

Essa padronização reduz a necessidade de tratamentos específicos.

5.12 Paginação e Filtros

As consultas que retornarem listas utilizarão parâmetros padronizados.

Exemplos de parâmetros:

- página;
- quantidade de registros;
- ordenação;
- direção da ordenação;
- filtros;
- termo de pesquisa.

O frontend será responsável por montar esses parâmetros e encaminhá-los aos serviços.

5.13 Upload de Arquivos

Quando necessário, o envio de arquivos utilizará requisições do tipo multipart/form-data.

Exemplos:

- anexos administrativos;
- documentos de campanhas;
- imagens de perfil;
- arquivos auxiliares.

Os componentes de upload deverão informar o progresso da operação e tratar falhas de forma amigável.

5.14 Download de Arquivos

Arquivos gerados pelo sistema, como relatórios e exportações, serão disponibilizados por endpoints específicos.

O frontend deverá:

- solicitar o recurso;
- acompanhar o progresso, quando aplicável;
- iniciar o download automaticamente;
- tratar erros de geração ou indisponibilidade.

5.15 Cancelamento de Requisições

Requisições poderão ser canceladas para evitar processamento desnecessário.

Situações típicas:

- troca rápida de filtros;
- navegação entre páginas;
- pesquisas em tempo real;
- fechamento de componentes.

Essa estratégia melhora a experiência do usuário e reduz carga no servidor.

5.16 Estratégia de Retry

O TanStack Query poderá repetir automaticamente determinadas operações.

Diretrizes:

- repetir apenas falhas temporárias;
- limitar a quantidade de tentativas;
- não repetir operações de escrita sem confirmação explícita;
- aplicar intervalo progressivo entre tentativas.

5.17 Timeout

Cada requisição possuirá um tempo máximo configurado.

Caso esse limite seja excedido:

- a operação será interrompida;
- o usuário será informado;
- o evento poderá ser registrado para monitoramento.

Os valores poderão variar conforme o tipo de operação.

5.18 Versionamento da API

A comunicação será preparada para múltiplas versões da API.

Exemplo de estrutura:

```
/api/v1/
```

```
/api/v2/
```

O frontend deverá concentrar a definição da versão em um único ponto de configuração, facilitando futuras migrações.

5.19 Segurança na Comunicação

Todas as comunicações deverão observar as seguintes diretrizes:

- utilização obrigatória de HTTPS em produção;

- envio do token apenas em conexões seguras;
- proteção contra interceptação de dados;
- validação de certificados;
- não exposição de informações sensíveis em URLs;
- tratamento seguro de erros retornados pela API.

5.20 Organização por Funcionalidade

Cada módulo manterá sua própria camada de comunicação.

Exemplo:

```
features/  
└─ territories/  
    └─ api/  
        └─ territoryApi.ts  
        └─ territoryQueries.ts  
        └─ territoryMutations.ts  
        └─ territoryKeys.ts  
    └─ hooks/  
    └─ services/
```

Essa organização favorece encapsulamento e facilita a evolução independente de cada funcionalidade.

5.21 Monitoramento das Requisições

As requisições poderão ser instrumentadas para fins de observabilidade.

Métricas sugeridas:

- tempo de resposta;
- quantidade de chamadas;
- falhas por endpoint;
- taxa de sucesso;
- operações mais acessadas.

Essas informações poderão ser integradas às ferramentas de monitoramento definidas na arquitetura do backend.

5.22 Relação com os Demais Documentos

Este capítulo complementa:

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a comunicação entre as camadas da aplicação.
Documento 06 – Arquitetura Backend	Especifica os serviços disponibilizados pelo backend.
Documento 08 – Segurança	Detalha autenticação, autorização e proteção da comunicação.
Documento 10 – API REST	Documenta os endpoints consumidos pelo frontend.

6 – Autenticação, Segurança e Controle de Acesso

6.1 Objetivo

Este capítulo estabelece a arquitetura de autenticação, autorização e segurança da camada frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é garantir que apenas usuários autenticados e autorizados tenham acesso às funcionalidades do sistema, assegurando confidencialidade, integridade e rastreabilidade das operações realizadas.

As definições apresentadas complementam o **Documento 06 – Arquitetura Backend** e serão detalhadas tecnicamente no **Documento 08 – Segurança**.

6.2 Princípios de Segurança

A implementação da segurança no frontend seguirá os seguintes princípios:

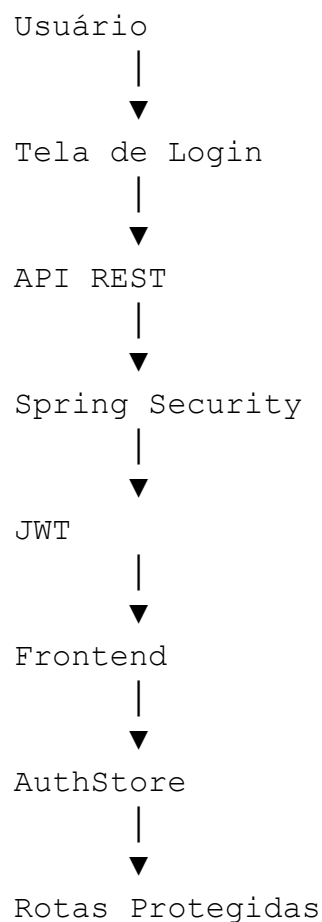
- autenticação centralizada;
- autorização baseada em perfis e permissões;
- menor privilégio (*Principle of Least Privilege*);
- proteção de rotas;
- validação contínua da sessão;

- tratamento uniforme de erros de autenticação;
- separação entre autenticação e autorização;
- não armazenamento de informações sensíveis em componentes.

Esses princípios reduzem riscos de acesso indevido e tornam a aplicação mais previsível.

6.3 Fluxo Geral de Autenticação

O processo de autenticação seguirá o fluxo abaixo:



Após a autenticação bem-sucedida, o usuário poderá acessar apenas as funcionalidades permitidas ao seu perfil.

6.4 Processo de Login

O login compreenderá as seguintes etapas:

1. preenchimento das credenciais;
2. validação inicial do formulário;
3. envio da requisição à API;
4. autenticação pelo backend;
5. geração do JWT;
6. retorno das informações do usuário;
7. armazenamento da sessão;
8. carregamento das permissões;
9. redirecionamento para o dashboard.

Em caso de falha, o usuário permanecerá na tela de autenticação com mensagens claras e objetivas.

6.5 Sessão do Usuário

A sessão deverá conter apenas as informações necessárias para o funcionamento da aplicação.

Exemplos:

- identificador do usuário;
- nome para exibição;
- perfis;
- permissões;
- congregação ativa;
- token JWT;
- horário de expiração da sessão.

Informações detalhadas do domínio deverão ser consultadas sob demanda por meio da API.

6.6 Gerenciamento da Sessão

O controle da sessão será responsabilidade da authStore.

Principais funções:

- iniciar sessão;
- atualizar dados do usuário;
- verificar autenticação;
- encerrar sessão;
- limpar dados locais;
- informar o estado da autenticação aos componentes.

A store será a única fonte de verdade para o estado de autenticação.

6.7 Proteção de Rotas

Todas as rotas privadas serão protegidas por componentes de controle de acesso.

Estrutura proposta:

```
routes/  
|  
├── PublicRoute.tsx  
├── PrivateRoute.tsx  
├── PermissionRoute.tsx  
├── RouteGuard.tsx  
└── AppRoutes.tsx
```

Cada componente terá uma responsabilidade específica na validação do acesso.

6.8 Rotas Públicas

As rotas públicas não exigirão autenticação.

Exemplos:

- login;
- recuperação de senha;
- redefinição de senha;

- página de acesso negado;
- páginas institucionais, se existirem.

Essas rotas permanecerão acessíveis mesmo sem sessão ativa.

6.9 Rotas Protegidas

As rotas protegidas exigirão autenticação válida.

Antes da renderização, serão verificadas:

- existência do token;
- validade da sessão;
- perfil do usuário;
- permissões necessárias.

Caso alguma condição não seja atendida, o acesso será negado.

6.10 Controle por Perfis

O sistema utilizará perfis para agrupar responsabilidades.

Perfis previstos:

Perfil	Finalidade
Administrador Global	Administração completa do sistema.
Administrador da Congregação	Gestão administrativa da congregação.
Ancião	Operações pastorais e administrativas autorizadas.
Servo Ministerial	Apoio às atividades administrativas.
Publicador	Operações permitidas ao publicador.
Usuário de Consulta	Acesso somente para leitura.

Os perfis serão atribuídos pelo backend e apenas interpretados pelo frontend.

6.11 Controle por Permissões

Além dos perfis, funcionalidades específicas poderão exigir permissões individuais.

Exemplos:

- criar territórios;
- editar publicações;
- cancelar pedidos;
- aprovar campanhas;
- visualizar auditorias;
- exportar relatórios.

Essa abordagem permite maior flexibilidade na administração dos acessos.

6.12 Hooks de Autorização

A verificação de acesso será encapsulada em hooks reutilizáveis.

Exemplos:

```
useAuth()  
usePermissions()  
useCurrentUser()  
useHasRole()  
useHasPermission()
```

Esses hooks facilitarão o controle de acesso nos componentes e páginas.

6.13 Tratamento de Sessões Expiradas

Quando o backend informar que a sessão expirou:

- o estado de autenticação será invalidado;
- os dados da sessão serão removidos;
- o usuário será redirecionado para o login;

- será exibida uma mensagem informando a necessidade de nova autenticação.

Esse comportamento será centralizado nos interceptadores HTTP.

6.14 Logout

O processo de encerramento da sessão compreenderá:

1. limpeza da authStore;
2. remoção do token;
3. limpeza do cache do TanStack Query;
4. remoção de dados temporários;
5. redirecionamento para a tela de login.

Esse fluxo evita que informações da sessão anterior permaneçam disponíveis.

6.15 Armazenamento Seguro

O frontend minimizará o armazenamento de dados sensíveis.

Diretrizes:

- evitar persistência desnecessária;
- limpar informações ao encerrar a sessão;
- não registrar tokens em logs;
- proteger preferências do usuário contra manipulação indevida;
- validar continuamente a autenticidade da sessão.

A estratégia definitiva de armazenamento será detalhada no Documento 08.

6.16 Controle de Navegação

O menu lateral e demais elementos de navegação serão construídos dinamicamente conforme as permissões do usuário.

Benefícios:

- simplificação da interface;

- redução de erros de navegação;
- ocultação de funcionalidades não autorizadas;
- melhor experiência do usuário.

6.17 Tratamento de Acesso Negado

Quando um usuário tentar acessar uma funcionalidade sem permissão, a aplicação deverá:

- impedir a navegação;
- registrar o evento, quando aplicável;
- exibir mensagem amigável;
- oferecer retorno seguro para uma área autorizada.

O sistema evitará revelar informações sobre recursos protegidos.

6.18 Segurança na Interface

Além do controle de acesso, a interface seguirá boas práticas de segurança.

Diretrizes:

- sanitização de dados exibidos;
- validação de entradas antes do envio;
- proteção contra manipulação indevida do estado da aplicação;
- não exposição de informações internas em mensagens de erro;
- remoção de dados sensíveis ao finalizar a sessão.

6.19 Integração com o Backend

O frontend atuará apenas como consumidor das decisões de segurança definidas pelo backend.

Responsabilidades do backend:

- autenticar usuários;
- emitir o JWT;

- validar permissões;
- autorizar operações.

Responsabilidades do frontend:

- armazenar a sessão;
- enviar o token nas requisições;
- proteger a navegação;
- adaptar a interface às permissões recebidas.

6.20 Relação com os Demais Componentes

A camada de autenticação interage com:

- AuthProvider;
- authStore;
- interceptadores do Axios;
- TanStack Query;
- componentes de navegação;
- PrivateRoute;
- PermissionRoute;
- layouts protegidos.

Essa integração garante comportamento uniforme em toda a aplicação.

6.21 Benefícios da Arquitetura

A arquitetura proposta proporciona:

- autenticação centralizada;
- autorização consistente;
- proteção de rotas;
- redução de duplicidade de código;
- facilidade para testes;

- escalabilidade;
- integração transparente com o backend;
- melhor experiência para o usuário.

6.22 Relação com os Demais Documentos

Este capítulo complementa:

Documento	Relação
Documento 03 – Regras de Negócio	Define as regras de acesso às funcionalidades.
Documento 04 – Casos de Uso	Especifica quais atores podem executar cada caso de uso.
Documento 06 – Arquitetura Backend	Define a autenticação e autorização no servidor.
Documento 08 – Segurança	Detalha os mecanismos de proteção adotados em toda a solução.

7 – Roteamento, Navegação e Layouts

7.1 Objetivo

Este capítulo define a arquitetura de roteamento e navegação do frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é estabelecer uma estrutura organizada para as rotas da aplicação, garantindo uma navegação consistente, segura e alinhada às permissões de cada usuário.

A solução será implementada utilizando **React Router DOM**, integrada ao sistema de autenticação, aos layouts da aplicação e ao gerenciamento de permissões.

7.2 Princípios

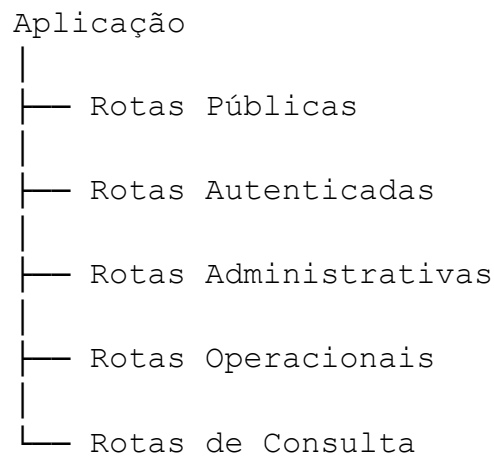
A arquitetura de navegação seguirá os seguintes princípios:

- separação entre rotas públicas e privadas;

- reutilização de layouts;
- carregamento sob demanda (*Lazy Loading*);
- proteção por autenticação e autorização;
- menus dinâmicos;
- URLs semânticas;
- navegação previsível;
- baixo acoplamento entre módulos.

7.3 Estrutura Geral de Rotas

A aplicação será organizada em grupos de rotas conforme sua finalidade.



Essa divisão facilita a aplicação de regras de segurança e a organização da navegação.

7.4 Organização dos Arquivos de Rotas

A configuração das rotas ficará centralizada no diretório routes.

```
routes/
|
|— AppRoutes.tsx
|— PublicRoutes.tsx
|— PrivateRoutes.tsx
|— AdminRoutes.tsx
|— RouteGuard.tsx
|— PermissionRoute.tsx
|— NotFoundRoute.tsx
|— index.ts
```

Cada arquivo terá uma responsabilidade específica, evitando concentrações excessivas de lógica.

7.5 Rotas Públicas

As rotas públicas estarão disponíveis independentemente da autenticação do usuário.

Exemplos:

Rota	Finalidade
/login	Autenticação do usuário
/forgot-password	Recuperação de senha
/reset-password	Redefinição de senha
/access-denied	Acesso negado
/404	Página não encontrada

Essas rotas utilizarão um layout simplificado.

7.6 Rotas Privadas

As rotas privadas exigirão autenticação válida.

Exemplos:

Rota	Módulo
<code>/dashboard</code>	Painel principal
<code>/territories</code>	Territórios
<code>/publications</code>	Publicações
<code>/orders</code>	Pedidos
<code>/campaigns</code>	Campanhas
<code>/reports</code>	Relatórios
<code>/settings</code>	Configurações

Antes da renderização, será verificada a validade da sessão.

7.7 Rotas Administrativas

Algumas funcionalidades serão restritas a perfis administrativos.

Exemplos:

```
/admin/users  
/admin/congregations  
/admin/roles  
/admin/permissions  
/admin/audit
```

O acesso será controlado por perfis e permissões definidos pelo backend.

7.8 Estrutura de URLs

As URLs deverão seguir um padrão consistente e orientado ao domínio.

Exemplos:

```
/territories
/territories/new
/territories/:id
/territories/:id/edit
/publications
/publications/:id
/orders
/orders/:id
/reports
```

As rotas deverão ser intuitivas e refletir claramente a funcionalidade acessada.

7.9 Navegação Hierárquica

A navegação respeitará a organização funcional do sistema.

Fluxo simplificado:

```
Dashboard
↓
Territórios
↓
Detalhes
↓
Editar
↓
Salvar
↓
Retornar à Lista
```

Esse padrão proporciona previsibilidade ao usuário.

7.10 Layouts da Aplicação

Os layouts serão responsáveis pela estrutura visual das diferentes áreas do sistema.

Estrutura proposta:

```
layouts/  
  MainLayout  
  DashboardLayout  
  AuthLayout  
  ReportLayout  
  EmptyLayout
```

Cada layout conterá apenas elementos estruturais, sem lógica de negócio.

7.11 MainLayout

O MainLayout será utilizado pela maior parte da aplicação.

Elementos previstos:

- menu lateral;
- barra superior;
- breadcrumb;
- área principal de conteúdo;
- rodapé;
- notificações globais.

Esse layout servirá como base para os módulos operacionais.

7.12 AuthLayout

Será utilizado nas páginas de autenticação.

Componentes:

- formulário centralizado;

- identidade visual;
- mensagens institucionais;
- suporte para recuperação de senha.

Não exibirá menus de navegação.

7.13 DashboardLayout

Especializado para a tela inicial do sistema.

Estrutura:

- indicadores;
- atalhos rápidos;
- atividades recentes;
- notificações;
- gráficos;
- calendário de atividades.

7.14 Breadcrumb

A navegação contextual será exibida por meio de breadcrumbs.

Exemplo:

```
Dashboard
>
Territórios
>
Território 015
>
Editar
```

Os breadcrumbs serão gerados automaticamente com base na configuração das rotas.

7.15 Menu Lateral

O menu principal será construído dinamicamente.

Critérios:

- perfil do usuário;
- permissões;
- congregação ativa;
- módulos habilitados.

Menus de funcionalidades não autorizadas não serão exibidos.

7.16 Barra Superior

A barra superior concentrará funcionalidades globais.

Elementos previstos:

- pesquisa rápida;
- notificações;
- troca de congregação;
- troca de idioma;
- perfil do usuário;
- acesso às configurações;
- logout.

7.17 Navegação Programática

Quando necessário, a navegação será realizada por meio de ações controladas pela aplicação.

Exemplos:

- conclusão de formulários;
- cancelamento de operações;
- autenticação bem-sucedida;

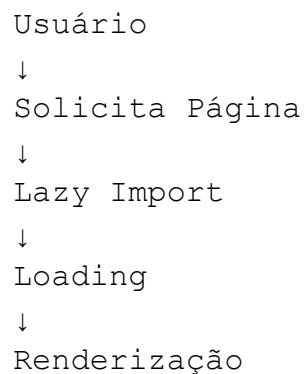
- encerramento da sessão;
- redirecionamentos por permissão.

Essa estratégia melhora a consistência da experiência do usuário.

7.18 Lazy Loading

As páginas serão carregadas sob demanda para reduzir o tempo de carregamento inicial.

Fluxo:



Enquanto o módulo é carregado, será exibido um componente de carregamento padronizado.

7.19 Página Não Encontrada

Rotas inexistentes direcionarão para uma página específica.

Conteúdo previsto:

- mensagem amigável;
- botão para retornar ao dashboard;
- link para a página inicial;
- registro opcional do evento para monitoramento.

7.20 Tratamento de Erros de Navegação

A navegação deverá tratar situações como:

- rota inexistente;
- sessão expirada;
- acesso negado;
- módulo indisponível;
- erro de carregamento.

Sempre será apresentada uma alternativa segura para continuidade da navegação.

7.21 Organização por Funcionalidade

Cada módulo poderá registrar suas próprias rotas.

Exemplo:

```
features/  
territories/  
routes/  
territory.routes.ts
```

Posteriormente, essas rotas serão agregadas ao conjunto principal da aplicação.

7.22 Fluxo Geral de Navegação

Login
↓
Dashboard
↓
Seleciona Módulo
↓
Lista
↓
Detalhes
↓
Operação
↓
Confirmação
↓
Retorno

Esse fluxo será reutilizado pela maioria das funcionalidades do sistema.

7.23 Acessibilidade na Navegação

A navegação deverá atender às boas práticas de acessibilidade.

Diretrizes:

- foco automático em mudanças de página;
- suporte completo à navegação por teclado;
- identificação adequada dos elementos;
- breadcrumbs acessíveis;
- menus compatíveis com leitores de tela.

7.24 Benefícios da Arquitetura

A arquitetura de navegação proporciona:

- organização clara das rotas;
- reutilização de layouts;
- melhor desempenho por meio de *Lazy Loading*;

- integração transparente com o sistema de autenticação;
- facilidade de manutenção;
- expansão simplificada de novos módulos.

7.25 Relação com os Demais Documentos

Documento	Relação
Documento 04 – Casos de Uso	Define os fluxos de navegação entre funcionalidades.
Documento 05 – Arquitetura do Sistema	Estabelece a comunicação entre frontend e backend.
Documento 06 – Arquitetura Backend	Fornecer os serviços consumidos pelas rotas protegidas.
Documento 08 – Segurança	Define as regras de autenticação e autorização aplicadas durante a navegação.

8 – Design System e Padronização Visual

8.1 Objetivo

O Design System do **Gestor de Territórios e Publicações (GTP)** tem como objetivo estabelecer um conjunto de padrões visuais, componentes reutilizáveis e diretrizes de experiência do usuário que garantam consistência em toda a aplicação.

Mais do que um conjunto de estilos, o Design System representa a linguagem visual oficial do sistema, permitindo que novas funcionalidades sejam desenvolvidas mantendo a mesma identidade, comportamento e qualidade de interação.

8.2 Princípios do Design System

O Design System será baseado nos seguintes princípios:

- consistência visual;

- simplicidade;
- clareza das informações;
- acessibilidade;
- reutilização de componentes;
- responsividade;
- escalabilidade;
- facilidade de manutenção.

Esses princípios orientarão todas as decisões relacionadas à interface do sistema.

8.3 Identidade Visual

A identidade visual do GTP deverá transmitir:

- organização;
- confiabilidade;
- simplicidade;
- modernidade;
- profissionalismo;
- facilidade de uso.

A interface utilizará elementos discretos, com foco na leitura das informações e na eficiência operacional.

8.4 Sistema de Cores

A paleta principal será composta por cores com funções bem definidas.

Categoria Finalidade

Primária Identidade do sistema, botões principais e elementos de destaque.

Secundária Ações complementares e elementos auxiliares.

Sucesso Confirmações e operações concluídas.

Categoria Finalidade

Atenção Avisos e situações que exigem atenção.

Erro Falhas, bloqueios e validações.

Informação Mensagens informativas e estados neutros.

Neutra Fundos, bordas, textos e divisores.

A implementação utilizará *tokens* de design, facilitando futuras alterações de identidade visual.

8.5 Tokens de Design

Todas as propriedades visuais serão centralizadas em tokens.

Categorias previstas:

- cores;
- tipografia;
- espaçamentos;
- bordas;
- sombras;
- raios de borda;
- opacidade;
- animações;
- pontos de quebra (*breakpoints*).

Essa abordagem garante uniformidade e facilita a manutenção.

8.6 Tipografia

A tipografia deverá priorizar legibilidade e hierarquia visual.

Hierarquia sugerida:

Elemento	Utilização
----------	------------

Título Principal	Identificação da página.
------------------	--------------------------

Título de Seção	Agrupamento de conteúdos.
-----------------	---------------------------

Subtítulo	Organização intermediária.
-----------	----------------------------

Texto Principal	Conteúdo e descrições.
-----------------	------------------------

Texto Auxiliar	Informações complementares.
----------------	-----------------------------

Legenda	Dados secundários e observações.
---------	----------------------------------

A tipografia será definida de forma consistente para todos os componentes.

8.7 Espaçamento

Será adotado um sistema de espaçamento baseado em múltiplos de uma unidade base.

Aplicações:

- margens;
- preenchimentos (*padding*);
- distância entre componentes;
- alinhamento de elementos;
- organização de formulários.

Essa padronização contribui para uma interface equilibrada e previsível.

8.8 Grid e Layout

O posicionamento dos elementos seguirá um sistema de grid responsivo.

Características:

- alinhamento consistente;
- distribuição uniforme dos componentes;
- adaptação automática para diferentes resoluções;
- organização visual das páginas.

O grid será utilizado tanto em telas simples quanto em dashboards complexos.

8.9 Ícones

Os ícones deverão seguir um padrão único em toda a aplicação.

Diretrizes:

- biblioteca padronizada;
- dimensões consistentes;
- significado claro;
- contraste adequado;
- uso moderado.

Os ícones complementarão o texto, evitando substituir informações essenciais.

8.10 Botões

Os botões serão classificados conforme sua importância.

Categorias:

- ação primária;
- ação secundária;
- ação terciária;
- botão de perigo;
- botão de confirmação;
- botão de cancelamento;
- botão somente com ícone.

Cada categoria possuirá aparência e comportamento padronizados.

8.11 Campos de Entrada

Os componentes de formulário seguirão um padrão único.

Exemplos:

- campo de texto;
- área de texto;
- seleção simples;
- seleção múltipla;
- caixa de seleção;
- botão de opção;
- seletor de data;
- seletor de hora;
- pesquisa;
- senha.

Todos deverão possuir:

- rótulo;
- indicação de obrigatoriedade;
- mensagem de ajuda;
- mensagem de erro;
- estado de carregamento;
- estado desabilitado.

8.12 Cartões (Cards)

Os cards serão utilizados para organizar informações em blocos.

Exemplos:

- indicadores do dashboard;
- resumo de campanhas;
- informações de territórios;

- dados de usuários;
- notificações;
- estatísticas.

Todos os cards seguirão o mesmo padrão visual.

8.13 Tabelas

As tabelas deverão apresentar:

- cabeçalhos destacados;
- ordenação;
- filtros;
- paginação;
- estados vazios;
- indicadores de carregamento;
- ações por linha;
- seleção múltipla quando aplicável.

A experiência deverá permanecer consistente em todos os módulos.

8.14 Modais e Diálogos

Os diálogos serão utilizados para:

- confirmações;
- exclusões;
- edições rápidas;
- visualizações detalhadas;
- alertas importantes.

Todos seguirão comportamento uniforme, incluindo foco inicial, fechamento e acessibilidade.

8.15 Mensagens de Feedback

As mensagens exibidas ao usuário serão classificadas em:

- sucesso;
- informação;
- aviso;
- erro.

Cada categoria possuirá aparência, ícone e comportamento consistentes.

8.16 Indicadores de Carregamento

Durante operações assíncronas, a interface deverá informar claramente o estado da aplicação.

Componentes previstos:

- indicador linear;
- indicador circular;
- *Skeleton Screens*;
- placeholders de carregamento.

Essa abordagem reduz a percepção de espera e melhora a experiência do usuário.

8.17 Temas

O Design System oferecerá suporte a múltiplos temas.

Inicialmente:

- tema claro;
- tema escuro.

A estrutura será preparada para permitir a inclusão de novos temas no futuro, caso necessário.

8.18 Responsividade

Todos os componentes deverão adaptar-se automaticamente aos diferentes dispositivos.

Diretrizes:

- menus recolhíveis;
- tabelas responsivas;
- formulários reorganizados;
- navegação otimizada para dispositivos móveis;
- componentes adaptativos.

A responsividade será tratada como requisito fundamental, não como adaptação posterior.

8.19 Acessibilidade

O Design System incorporará requisitos de acessibilidade desde sua concepção.

Diretrizes:

- contraste adequado;
- navegação por teclado;
- foco visível;
- textos alternativos;
- compatibilidade com leitores de tela;
- componentes semânticos;
- áreas clicáveis adequadas.

Essas práticas contribuem para uma aplicação inclusiva e em conformidade com padrões reconhecidos.

8.20 Animações e Transições

As animações terão papel funcional, reforçando mudanças de estado sem comprometer o desempenho.

Exemplos:

- abertura de menus;
- exibição de modais;
- carregamento de conteúdo;
- notificações;
- expansão de painéis.

As transições deverão ser discretas, consistentes e rápidas.

8.21 Padronização Visual

Toda a aplicação deverá seguir regras comuns de:

- alinhamento;
- espaçamento;
- tipografia;
- nomenclatura de componentes;
- cores;
- estados visuais;
- comportamento interativo.

Essa uniformidade reduz a curva de aprendizado e fortalece a identidade do sistema.

8.22 Governança do Design System

O Design System será tratado como um ativo arquitetural do projeto.

Diretrizes:

- documentação centralizada;
- versionamento;
- revisão de novos componentes;
- controle de alterações;
- reutilização obrigatória antes da criação de novos elementos.

Essa governança evita inconsistências ao longo da evolução do sistema.

8.23 Relação com os Demais Documentos

Documento	Relação
Documento 02 – Requisitos Funcionais	Define as interfaces necessárias para atender às funcionalidades.
Documento 04 – Casos de Uso	Especifica os fluxos que serão representados visualmente.
Documento 05 – Arquitetura do Sistema	Alinha a arquitetura visual à arquitetura geral da solução.
Documento 09 – UX/UI	Detalhará wireframes, protótipos e fluxos baseados neste Design System.

9 – Formulários, Validação e Experiência do Usuário

9.1 Objetivo

Este capítulo estabelece a arquitetura dos formulários e das interações do usuário no **Gestor de Territórios e Publicações (GTP)**.

Seu objetivo é definir padrões para criação, validação e processamento de formulários, garantindo consistência visual, integridade dos dados e uma experiência de uso intuitiva.

Todos os formulários da aplicação seguirão uma arquitetura comum, baseada em componentes reutilizáveis, validação declarativa e integração com as regras de negócio do backend.

9.2 Princípios

Os formulários deverão seguir os seguintes princípios:

- simplicidade;
- consistência;
- acessibilidade;
- prevenção de erros;
- feedback imediato;

- validação em múltiplas camadas;
- reutilização de componentes;
- baixo acoplamento.

Esses princípios visam reduzir erros de preenchimento e aumentar a eficiência das operações.

9.3 Arquitetura dos Formulários

Todos os formulários serão estruturados em camadas bem definidas.

```
Página
↓
Componente do Formulário
↓
React Hook Form
↓
Schema Zod
↓
Serviço
↓
API REST
↓
Spring Boot
```

Cada camada possui responsabilidades específicas, facilitando testes e manutenção.

9.4 React Hook Form

O gerenciamento dos formulários será realizado pelo **React Hook Form**.

Principais responsabilidades:

- controle dos campos;
- gerenciamento do estado do formulário;
- validação;
- envio dos dados;
- reinicialização;
- controle de alterações;
- desempenho otimizado.

A utilização dessa biblioteca reduz renderizações e simplifica a implementação.

9.5 Validação com Zod

Todas as validações do frontend serão definidas utilizando **Zod**.

Os esquemas deverão representar:

- campos obrigatórios;
- formatos válidos;
- limites de tamanho;
- valores mínimos;
- valores máximos;
- expressões regulares;
- tipos de dados.

A validação será executada antes do envio da requisição ao backend.

9.6 Camadas de Validação

A validação ocorrerá em diferentes níveis.

Camada	Responsabilidade
Interface	Validação imediata do preenchimento.
React Hook Form	Controle do estado e erros do formulário.
Zod	Regras estruturais e sintáticas.
Backend	Regras de negócio e consistência dos dados.

Essa abordagem evita inconsistências e reduz requisições inválidas.

9.7 Estrutura dos Componentes de Formulário

Os componentes serão reutilizados em toda a aplicação.

```
components/  
└─ form/  
    └─ Form.tsx  
    └─ FormField.tsx  
    └─ FormLabel.tsx  
    └─ FormError.tsx  
    └─ FormActions.tsx  
    └─ FormSection.tsx  
    └─ RequiredIndicator.tsx  
    └─ HelpText.tsx
```

Essa organização promove consistência e reduz duplicação de código.

9.8 Tipos de Campos

O sistema disponibilizará componentes específicos para diferentes tipos de entrada.

Exemplos:

- texto;
- número;
- moeda;
- telefone;
- e-mail;
- senha;
- data;
- hora;
- seleção simples;
- seleção múltipla;
- caixa de seleção;
- botão de opção;
- upload de arquivos;
- pesquisa;
- editor de texto, quando necessário.

Cada componente será responsável por sua própria apresentação e comportamento.

9.9 Máscaras e Formatação

Campos que exigem formatos específicos utilizarão máscaras apropriadas.

Exemplos:

- telefone;
- CEP;
- CPF/CNPJ, quando aplicável;
- datas;
- valores monetários;
- porcentagens.

A formatação ocorrerá apenas na interface, preservando o formato esperado pela API.

9.10 Mensagens de Validação

As mensagens deverão ser:

- objetivas;
- compreensíveis;

- específicas;
- consistentes em toda a aplicação.

Exemplos:

- campo obrigatório;
- formato inválido;
- valor fora do intervalo permitido;
- informação duplicada;
- seleção obrigatória.

As mensagens deverão orientar o usuário sobre como corrigir o problema.

9.11 Feedback Durante o Preenchimento

Os formulários fornecerão retorno visual em tempo real.

Estados previstos:

- campo vazio;
- campo válido;
- campo inválido;
- carregando;
- desabilitado;
- somente leitura.

Esses estados deverão ser claramente identificáveis.

9.12 Organização dos Formulários

Formulários extensos serão divididos em seções.

Exemplo:

```
Dados Gerais
↓
Informações Complementares
↓
Configurações
↓
Confirmação
```

Essa divisão melhora a legibilidade e reduz a sobrecarga cognitiva.

9.13 Botões de Ação

Os formulários utilizarão ações padronizadas.

Botões previstos:

- salvar;
- cancelar;
- limpar;
- excluir;
- voltar;
- confirmar;
- avançar;
- concluir.

A posição e o comportamento desses botões deverão permanecer consistentes.

9.14 Confirmação de Operações

Operações críticas exigirão confirmação explícita.

Exemplos:

- exclusão;
- cancelamento;
- encerramento de campanhas;
- redefinição de configurações;
- remoção de usuários.

A confirmação reduzirá o risco de ações acidentais.

9.15 Prevenção de Perda de Dados

Caso o usuário tente abandonar uma página com alterações não salvas, a aplicação deverá alertá-lo.

Situações monitoradas:

- fechamento da aba;
- troca de rota;
- atualização da página;
- navegação externa.

Essa funcionalidade protege o trabalho realizado pelo usuário.

9.16 Processamento Assíncrono

Durante o envio de formulários:

- os campos poderão ser desabilitados;
- será exibido indicador de carregamento;
- o botão de envio evitará múltiplos cliques;
- mensagens de progresso poderão ser apresentadas.

Isso evita duplicidade de operações.

9.17 Tratamento de Erros

Os erros poderão ser classificados em:

- validação local;
- validação do backend;
- falhas de comunicação;
- conflitos de negócio;
- indisponibilidade do serviço.

Cada categoria possuirá tratamento específico e mensagens adequadas.

9.18 Experiência do Usuário

As interações deverão seguir princípios de UX.

Diretrizes:

- minimizar quantidade de cliques;
- reduzir digitação desnecessária;
- preencher dados automaticamente quando possível;
- priorizar informações relevantes;
- evitar interrupções desnecessárias;
- fornecer feedback imediato.

O foco será tornar as operações rápidas e intuitivas.

9.19 Acessibilidade

Os formulários deverão atender aos requisitos de acessibilidade.

Incluem:

- associação correta entre rótulos e campos;
- navegação por teclado;
- foco visível;
- mensagens de erro acessíveis;
- identificação de campos obrigatórios;
- compatibilidade com leitores de tela.

9.20 Reutilização

Os formulários compartilharão componentes, validações e comportamentos.

Benefícios:

- menor duplicação de código;
- manutenção simplificada;
- experiência uniforme;
- redução de inconsistências.

9.21 Integração com o Backend

O frontend validará aspectos estruturais dos dados, enquanto o backend permanecerá responsável pelas regras de negócio.

Fluxo:

```
Usuário
↓
Validação Local
↓
Zod
↓
React Hook Form
↓
API REST
↓
Spring Validation
↓
Regras de Negócio
```

Essa divisão garante robustez e evita duplicação de responsabilidades.

9.22 Relação com os Demais Documentos

Documento	Relação
Documento 02 – Requisitos Funcionais	Define os dados coletados pelos formulários.
Documento 03 – Regras de Negócio	Estabelece as validações de domínio.
Documento 04 – Casos de Uso	Especifica os fluxos de interação do usuário.
Documento 06 – Arquitetura Backend	Define as validações executadas no servidor.

9.23 Benefícios da Arquitetura

A arquitetura proposta proporciona:

- formulários consistentes;
- validações previsíveis;
- reutilização de componentes;
- melhor desempenho;
- redução de erros;
- experiência do usuário aprimorada;
- facilidade de manutenção;
- alinhamento entre frontend e backend.

10 – Testes, Performance, Observabilidade e Diretrizes de Implementação

10.1 Objetivo

Este capítulo estabelece as diretrizes para garantir a qualidade, desempenho, confiabilidade e manutenção do frontend do **Gestor de Territórios e Publicações (GTP)**.

Seu propósito é definir padrões para:

- testes automatizados;
- monitoramento da aplicação;
- otimização de desempenho;
- organização do código;

- qualidade do desenvolvimento;
- processo de entrega contínua.

Essas diretrizes deverão ser seguidas durante todo o ciclo de vida do projeto.

10.2 Estratégia Geral de Qualidade

A qualidade do frontend será baseada em múltiplas camadas de verificação.

```
Desenvolvedor
↓
ESLint
↓
Prettier
↓
TypeScript
↓
Testes Unitários
↓
Testes de Integração
↓
Testes E2E
↓
Build
↓
Deploy
```

Cada camada reduz a probabilidade de defeitos chegarem ao ambiente de produção.

10.3 Testes Unitários

Os testes unitários validarão componentes, hooks e funções utilitárias de forma isolada.

Ferramenta adotada:

- **Vitest**

Elementos testáveis:

- componentes reutilizáveis;
- funções utilitárias;

- máscaras;
- validadores;
- hooks customizados;
- stores do Zustand;
- transformações de dados.

Objetivos:

- validar regras de funcionamento;
- prevenir regressões;
- facilitar refatorações.

10.4 Testes de Componentes

Os componentes React serão testados utilizando:

- **Testing Library**

Serão verificadas:

- renderização correta;
- acessibilidade;
- interação do usuário;
- estados visuais;
- mensagens de erro;
- comportamento em diferentes cenários.

Os testes deverão reproduzir a interação do usuário sempre que possível.

10.5 Testes de Integração

Os testes de integração validarão a comunicação entre diferentes partes da aplicação.

Exemplos:

- formulários e validações;
- integração com TanStack Query;

- navegação entre páginas;
- interação entre componentes;
- gerenciamento de estado.

O objetivo é garantir que os módulos funcionem corretamente quando utilizados em conjunto.

10.6 Testes End-to-End (E2E)

Os fluxos críticos do sistema serão automatizados utilizando:

- **Playwright**

Cenários prioritários:

- autenticação;
- recuperação de senha;
- gerenciamento de territórios;
- distribuição de publicações;
- geração de relatórios;
- administração de usuários;
- configurações.

Esses testes simularão o comportamento real dos usuários.

10.7 Cobertura de Testes

A cobertura mínima recomendada será:

Categoria	Meta
Componentes compartilhados	≥ 90%
Hooks	≥ 90%
Utilitários	≥ 95%
Stores	≥ 90%

Categoria	Meta
Serviços	≥ 85%
Funcionalidades críticas	100% dos fluxos principais

Mais importante que a porcentagem será a cobertura dos comportamentos essenciais.

10.8 Performance da Aplicação

A arquitetura deverá priorizar desempenho desde o início do desenvolvimento.

Diretrizes:

- minimizar renderizações;
- reutilizar componentes;
- evitar estados duplicados;
- reduzir chamadas HTTP;
- utilizar cache inteligente;
- carregar recursos sob demanda.

A otimização será tratada como parte da arquitetura, e não como atividade posterior.

10.9 Lazy Loading

Todos os módulos independentes deverão utilizar carregamento sob demanda.

Exemplos:

- relatórios;
- administração;
- configurações;
- auditoria.

Benefícios:

- menor tempo de carregamento inicial;

- redução do tamanho do bundle;
- melhor experiência do usuário.

10.10 Code Splitting

A aplicação utilizará divisão automática do código.

Objetivos:

- reduzir o JavaScript carregado inicialmente;
- otimizar o cache do navegador;
- permitir atualização independente de módulos.

10.11 Memoização

A memoização será utilizada apenas quando houver benefício comprovado.

Recursos disponíveis:

- `React.memo`;
- `useMemo`;
- `useCallback`.

Seu uso indiscriminado deverá ser evitado para não aumentar a complexidade sem ganhos reais.

10.12 Otimização de Renderização

Boas práticas:

- listas com chaves estáveis;
- virtualização para grandes volumes de dados;
- componentes menores e especializados;
- separação entre lógica e apresentação;
- atualização apenas dos elementos afetados.

Essas práticas reduzem o consumo de recursos e melhoram a responsividade da interface.

10.13 Observabilidade

O frontend deverá fornecer informações úteis para diagnóstico e monitoramento.

Eventos sugeridos:

- erros de navegação;
- falhas de autenticação;
- exceções não tratadas;
- tempo de carregamento;
- falhas de comunicação com a API;
- ações críticas do usuário.

Esses eventos poderão ser integrados às ferramentas de observabilidade adotadas pelo projeto.

10.14 Tratamento Global de Erros

A aplicação utilizará um componente de captura global de erros (*Error Boundary*).

Responsabilidades:

- impedir falhas em cascata;
- registrar exceções;
- exibir mensagens amigáveis;
- permitir recuperação quando possível.

Erros críticos deverão ser registrados para análise posterior.

10.15 Qualidade do Código

Todo o código deverá atender aos seguintes critérios:

- tipagem completa;
- ausência de código duplicado;
- funções pequenas;
- componentes especializados;

- nomenclatura consistente;
- comentários apenas quando necessários;
- documentação das partes complexas.

A legibilidade será considerada requisito arquitetural.

10.16 Padronização de Código

Serão adotadas ferramentas de padronização.

Ferramentas:

- ESLint;
- Prettier;
- EditorConfig.

Objetivos:

- reduzir divergências entre desenvolvedores;
- manter estilo uniforme;
- facilitar revisões de código.

10.17 Versionamento

A evolução do frontend seguirá uma estratégia baseada em ramificações controladas.

Diretrizes:

- desenvolvimento em branches específicas;
- revisões por Pull Request;
- integração contínua;
- versionamento semântico;
- histórico de alterações documentado.

10.18 Integração Contínua

O pipeline de integração deverá executar automaticamente:

1. instalação das dependências;
2. análise estática;
3. compilação TypeScript;
4. testes unitários;
5. testes de integração;
6. geração do build.

Somente versões aprovadas seguirão para publicação.

10.19 Deploy Contínuo

O processo de implantação deverá contemplar:

- ambiente de desenvolvimento;
- homologação;
- produção.

Cada ambiente possuirá configurações independentes e variáveis específicas.

10.20 Documentação Técnica

A arquitetura deverá permanecer sincronizada com a implementação.

Sempre que novos módulos forem adicionados, deverão ser atualizados:

- documentação da arquitetura;
- componentes do Design System;
- diagramas relevantes;
- documentação da API, quando aplicável.

Essa prática reduz divergências entre documentação e código.

10.21 Checklist para Novas Funcionalidades

Toda nova funcionalidade deverá atender ao seguinte checklist:

- requisitos funcionais implementados;
- regras de negócio respeitadas;
- componentes reutilizados;
- validações implementadas;
- testes automatizados criados;
- acessibilidade verificada;
- desempenho avaliado;
- documentação atualizada.

10.22 Evolução da Arquitetura

A arquitetura foi concebida para permitir:

- novos módulos;
- novos perfis de usuário;
- novos idiomas;
- novos temas visuais;
- novos componentes;
- novas integrações;
- novas funcionalidades sem necessidade de reorganização estrutural.

Essa flexibilidade assegura a longevidade do projeto.

10.23 Relação com os Demais Documentos

Documento	Relação
Documento 05 – Arquitetura do Sistema	Define a arquitetura geral da solução.

Documento	Relação
Documento 06 – Arquitetura Backend	Estabelece a integração entre frontend e backend.
Documento 08 – Segurança	Complementa os mecanismos de proteção da aplicação.
Documento 09 – Docker	Define o ambiente de execução e implantação.
Documento 10 – API REST	Especifica os contratos consumidos pelo frontend.

11 – Conclusão da Arquitetura Frontend

11.1 Síntese da Arquitetura

A arquitetura frontend do **Gestor de Territórios e Publicações (GTP)** foi concebida para atender aos requisitos funcionais, regras de negócio e diretrizes arquiteturais definidas nos documentos anteriores. A adoção de **React 19**, **TypeScript**, **Vite**, **Tailwind CSS**, **TanStack Query**, **Zustand** e **React Hook Form** estabelece uma base tecnológica moderna, modular e preparada para evolução contínua.

11.2 Benefícios Esperados

A arquitetura proposta oferece:

- alta modularidade;
- baixo acoplamento entre funcionalidades;
- reutilização intensiva de componentes;
- integração consistente com o backend;
- facilidade de testes;
- manutenção simplificada;
- desempenho otimizado;
- escalabilidade para novos módulos;
- experiência de usuário consistente;
- aderência às boas práticas do ecossistema React.

11.3 Alinhamento com a Arquitetura Geral

Este documento está diretamente alinhado aos demais artefatos da solução:

- **Documento 02 – Requisitos Funcionais:** define as funcionalidades implementadas pela interface.
- **Documento 03 – Regras de Negócio:** orienta as validações e comportamentos da aplicação.
- **Documento 04 – Casos de Uso:** descreve os fluxos de interação do usuário.
- **Documento 05 – Arquitetura do Sistema:** estabelece a visão arquitetural global.
- **Documento 06 – Arquitetura Backend:** define os serviços consumidos pelo frontend.
- **Documento 08 – Segurança:** complementa os mecanismos de proteção da aplicação.
- **Documento 10 – API REST:** documenta os contratos utilizados pela camada de serviços.

11.4 Diretrizes para Evolução

A evolução do frontend deverá preservar os princípios arquiteturais estabelecidos neste documento, garantindo:

- consistência estrutural;
- reutilização de componentes;
- documentação atualizada;
- conformidade com o Design System;
- manutenção da cobertura de testes;
- compatibilidade com futuras versões da plataforma.

11.5 Considerações Finais

A arquitetura apresentada fornece uma base sólida para o desenvolvimento do frontend do GTP, permitindo que a aplicação evolua de forma organizada, segura e

sustentável. Sua organização por funcionalidades, a separação clara de responsabilidades e o uso de tecnologias modernas favorecem a produtividade da equipe e a qualidade do software entregue.